

main.dart

```
> main.dart > ...
1 import 'package:flutter/material.dart';
2
3 import 'views/login/login_view.dart';
4
5 Run | Debug | Profile
6 void main() {
7   runApp(MaterialApp(home: LoginView()));
8 }
```

utils.dart

```
1 class Utils {
2   static String ip_servidor = '192.168.1.215';
3 }
4
```

login_view.dart

```
3 import 'dart:convert';
4
5 import 'package:flutter/material.dart';
6 import 'package:funcionarios_autentic_json/services/utils.dart';
7 import 'package:funcionarios_autentic_json/views/home/home_view.dart';
8 import 'package:funcionarios_autentic_json/views/login/cadastra_usuario_view.dart';
9 import 'package:funcionarios_autentic_json/views/login/recupera_senha_view.dart';
10 import 'package:http/http.dart' as http;
11
12 class LoginView extends StatefulWidget {
13   const LoginView({super.key});
14
15   @override
16   State<LoginView> createState() => _LoginViewState();
17 }
18
19 class _LoginViewState extends State<LoginView> {
20   TextEditingController emailController = TextEditingController();
21   TextEditingController senhaController = TextEditingController();
22
23   String ip_servidor = Utils.ip_servidor;
24
25   Future<void> autenticaUsuario(Map<String, dynamic> mapLogin) async {
26     print("MAPA LOGIN: " + mapLogin.toString());
27     var response = await http.post(
28       Uri.parse('http://$ip_servidor:5000/loginUsuario'),
29       headers: {
30         'Accept': 'application/json',
31         'content-type': 'application/json',
32       },
33       body: jsonEncode(mapLogin),
34       encoding: Encoding.getByName('utf-8'),
35     );
36
37     if (response.statusCode == 200) {
38       // Cadastro bem-sucedido
39       Map<String, dynamic> responseDadosUsuario = jsonDecode(response.body);
40       print("DADOS LOGIN: " + responseDadosUsuario.toString());
41       ScaffoldMessenger.of(context).showSnackBar(
42         SnackBar(
43           content: Text(
44             responseDadosUsuario["response"]["mensagem"].toString(),
45           ), // Text
46         ), // SnackBar
47       );
48       Navigator.pushReplacement(
49         context,
50         MaterialPageRoute(
51           builder: (context) => const HomeView(),
52           settings: RouteSettings(arguments: responseDadosUsuario["response"]),
53         ), // MaterialPageRoute
54       );
55     }
56   }
57 }
```

```

53     ), // MaterialPageRoute
54   );
55   print('Usuário cadastrado com sucesso!');
56 } else {
57   // Erro no cadastro
58   Map<String, dynamic> responseDadosUsuario = jsonDecode(response.body);
59   ScaffoldMessenger.of(context).showSnackBar(
60     SnackBar(
61       content: Text(
62         responseDadosUsuario["response"]["mensagem"].toString(),
63       ), // Text
64     ), // SnackBar
65   );
66   print('Erro ao cadastrar usuário: ${response.body}');
67 }
68 }
69
70 @override
71 Widget build(BuildContext context) {
72   return Scaffold(
73     body: Container(
74       padding: const EdgeInsets.all(16),
75       color: Colors.blue[100],
76       child: Column(
77         mainAxisAlignment: MainAxisAlignment.center,
78         children: [
79           Text(
80             'GestãoFlix',
81             style: TextStyle(
82               fontSize: 32,
83               fontWeight: FontWeight.bold,
84               color: Colors.blueGrey[800],
85             ), // TextStyle
86           ), // Text
87           Text(
88             'Bem-vindo! Faça login para continuar.',
89             style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
90           ), // Text
91           const SizedBox(height: 32),
92           TextField(
93             controller: emailController,
94             decoration: const InputDecoration(
95               fillColor: Colors.white,
96               filled: true,
97               prefixIcon: Icon(Icons.email, color: Colors.amber),
98               labelText: 'Email',
99               border: OutlineInputBorder(),
100           ), // InputDecoration
101           ), // TextField
102           const SizedBox(height: 16),

```

```

102     const SizedBox(height: 16),
103     TextField(
104       obscureText: true,
105       controller: senhaController,
106       decoration: const InputDecoration(
107         fillColor: Colors.white,
108         filled: true,
109         prefixIcon: Icon(Icons.lock, color: Colors.amber),
110         labelText: 'Senha',
111         border: OutlineInputBorder(),
112       ), // InputDecoration
113     ), // TextField
114     const SizedBox(height: 16),
115     ElevatedButton(
116       onPressed: () {
117         // Lógica de autenticação aqui
118         Map<String, dynamic> mapLogin = {
119           "emailUsuario": emailController.text,
120           "senhaUsuario": senhaController.text,
121         };
122         print("DADOS LOGIN: " + mapLogin.toString()); // Use interpol
123         autenticaUsuario(mapLogin);
124       },
125       child: const Text('Entrar'),
126     ), // ElevatedButton
127     SizedBox(height: 16),
128     TextButton(
129       child: const Text('Esqueci minha senha'),
130       onPressed: () {
131         Navigator.push(
132           context,
133           MaterialPageRoute(
134             builder: (context) => const RecuperaSenhaView(),
135           ), // MaterialPageRoute
136         );
137       },
138     ), // TextButton
139     SizedBox(height: 28),
140     ElevatedButton(
141       child: const Text('Criar nova conta'),
142       onPressed: () {
143         // Lógica para criar nova conta aqui
144         Navigator.push(
145           context,
146           MaterialPageRoute(
147             builder: (context) => const CadastraUsuarioView(),
148           ), // MaterialPageRoute
149         );
150       },
151     ), // ElevatedButton
152   ],
153 ), // Column
154 ), // Container
155 ); // Scaffold
156 }
157 }
158

```

cadastra_usuario_view.dart

```
3 import 'dart:convert';
4
5 import 'package:flutter/material.dart';
6 import 'package:funcionarios_autentic_json/services/Utils.dart';
7 import 'package:http/http.dart' as http;
8
9 class CadastraUsuarioView extends StatefulWidget {
10   const CadastraUsuarioView({super.key});
11
12   @override
13   State<CadastraUsuarioView> createState() => _CadastraUsuarioViewState();
14 }
15
16 class _CadastraUsuarioViewState extends State<CadastraUsuarioView> {
17   TextEditingController nomeController = TextEditingController();
18   TextEditingController emailController = TextEditingController();
19   TextEditingController senhaController = TextEditingController();
20
21   String ip_servidor = Utils.ip_servidor;
22
23   // Função para enviar os dados via json na rota /cadastraUsuario
24   Future<void> cadastrarUsuario(Map<String, dynamic> mapUsuario) async {
25     print("MAPA USUARIO: " + mapUsuario.toString());
26     var response = await http.post(
27       Uri.parse('http://$ip_servidor:5000/cadastraUsuario'),
28       headers: {
29         'Accept': 'application/json',
30         'content-type': 'application/json',
31       },
32       body: jsonEncode(mapUsuario),
33       encoding: Encoding.getByName('utf-8'),
34     );
35
36     if (response.statusCode == 200) {
37       // Cadastro bem-sucedido
38       ScaffoldMessenger.of(
39         context,
40       ).showSnackBar(SnackBar(content: Text("Cadastrado com sucesso!!")));
41       Navigator.pop(context);
42       print('Usuário cadastrado com sucesso!');
43     } else {
44       // Erro no cadastro
45       print('Erro ao cadastrar usuário: ${response.body}');
46     }
47   }
48
49   @override
50   Widget build(BuildContext context) {
51     return Scaffold(
52       appBar: AppBar(
53         title: const Text('Cadastro de Usuário'),
54         backgroundColor: Colors.blueGrey[800],
55         foregroundColor: Colors.amber,
56       ), // AppBar
57       body: Container(
58         padding: const EdgeInsets.all(16),
59         color: Colors.blue[100],
60         child: Column(
61           mainAxisAlignment: MainAxisAlignment.center,
62           children: [
63             Text(
64               'Crie sua conta',
```

```

64     'Crie sua conta',
65     style: TextStyle(
66       fontSize: 32,
67       fontWeight: FontWeight.bold,
68       color: Colors.blueGrey[800],
69     ), // TextStyle
70   ), // Text
71   const SizedBox(height: 32),
72   TextField(
73     decoration: const InputDecoration(
74       fillColor: Colors.white,
75       filled: true,
76       prefixIcon: Icon(Icons.person, color: Colors.amber),
77       labelText: 'Nome Completo',
78       border: OutlineInputBorder(),
79     ), // InputDecoration
80     controller: nomeController,
81   ), // TextField
82   const SizedBox(height: 16),
83   TextField(
84     decoration: const InputDecoration(
85       fillColor: Colors.white,
86       filled: true,
87       prefixIcon: Icon(Icons.email, color: Colors.amber),
88       labelText: 'Email',
89       border: OutlineInputBorder(),
90     ), // InputDecoration
91     controller: emailController,
92   ), // TextField
93   const SizedBox(height: 16),
94   TextField(
95     obscureText: true,
96     decoration: const InputDecoration(
97       fillColor: Colors.white,
98       filled: true,
99       prefixIcon: Icon(Icons.lock, color: Colors.amber),
100      labelText: 'Senha',
101      border: OutlineInputBorder(),
102    ), // InputDecoration
103    controller: senhaController,
104  ), // TextField
105  const SizedBox(height: 16),
106  ElevatedButton(
107    onPressed: () {
108      // Lógica de cadastro aqui
109      String nome = nomeController.text;
110      String email = emailController.text;
111      String senha = senhaController.text;
112
113      Map<String, dynamic> mapUsuario = {
114        "nomeUsuario": nome,
115        "emailUsuario": email,
116        "senhaUsuario": senha,
117      };
118
119      cadastrarUsuario(mapUsuario);
120    },
121    child: const Text('Cadastrar'),
122  ), // ElevatedButton
123 ],
124 ), // Column
125 ), // Container
126 ); // Scaffold
127 }
128
129

```

recupera_senha_view.dart

```
5 import 'package:flutter/material.dart';
6 import 'package:funcionarios_autentic_json/services/utils.dart';
7 import 'package:funcionarios_autentic_json/views/login/tela_nova_senha_view.dart';
8 import 'package:http/http.dart' as http;
9
10 class RecuperaSenhaView extends StatefulWidget {
11   const RecuperaSenhaView({super.key});
12
13   @override
14   State<RecuperaSenhaView> createState() => _RecuperaSenhaViewState();
15 }
16
17 class _RecuperaSenhaViewState extends State<RecuperaSenhaView> {
18   String ip_servidor = Utils.ip_servidor;
19
20   TextEditingController emailController = TextEditingController();
21
22   Future<void> recuperaSenha(String emailUsuario) async {
23     Map<String, dynamic> mapEmail = {"emailUsuario": emailUsuario};
24
25     var response = await http.post(
26       Uri.parse('http://$ip_servidor:5000/recuperaSenha'),
27       headers: {
28         'Accept': 'application/json',
29         'content-type': 'application/json',
30       },
31       body: jsonEncode(mapEmail),
32       encoding: Encoding.getByName('utf-8'),
33     );
34     var dados = jsonDecode(response.body)[0];
35     print("DADOS:" + dados.toString());
36     if (response.statusCode == 200) {
37       Navigator.push(
38         context,
39         MaterialPageRoute(
40           builder: (context) => const TelaNovaSenha(),
41           settings: RouteSettings(arguments: emailController.text),
42         ), // MaterialPageRoute
43       );
44       // Cadastro bem-sucedido
45       ScaffoldMessenger.of(
46         context,
47       ).showSnackBar(SnackBar(content: Text("E-mail enviado")));
48     } else {
49       ScaffoldMessenger.of(
50         context,
51       ).showSnackBar(SnackBar(content: Text("Erro ao enviar o e-mail")));
52     }
53   }
54 }
```

```

54
55 @override
56 Widget build(BuildContext context) {
57   return Scaffold(
58     appBar: AppBar(
59       title: const Text('Recuperação de Senha'),
60       backgroundColor: Colors.blueGrey[800],
61       foregroundColor: Colors.amber,
62     ), // AppBar
63     body: Container(
64       padding: const EdgeInsets.all(16),
65       color: Colors.blue[100],
66       child: Column(
67         mainAxisAlignment: MainAxisAlignment.center,
68         children: [
69           Text(
70             'Recuperação de Senha',
71             style: TextStyle(
72               fontSize: 32,
73               fontWeight: FontWeight.bold,
74               color: Colors.blueGrey[800],
75             ), // TextStyle
76           ), // Text
77           const SizedBox(height: 32),
78           TextField(
79             controller: emailController,
80             decoration: const InputDecoration(
81               fillColor: Colors.white,
82               filled: true,
83               prefixIcon: Icon(Icons.email, color: Colors.amber),
84               labelText: 'Digite seu email',
85               border: OutlineInputBorder(),
86             ), // InputDecoration
87           ), // TextField
88           const SizedBox(height: 16),
89           ElevatedButton(
90             onPressed: () {
91               // Lógica de recuperação de senha aqui
92               recuperaSenha(emailController.text);
93               print("Envia código de recuperação para o email");
94             },
95             child: const Text('Enviar código de recuperação'),
96           ), // ElevatedButton
97         ],
98       ), // Column
99     ), // Container
100   ); // Scaffold
101 }
102 }

```

tela_nova_senha_view.dart

```
3 import 'dart:convert';
4
5 import 'package:flutter/material.dart';
6 import 'package:funcionarios_autentic_json/services/utils.dart';
7 import 'package:funcionarios_autentic_json/views/login/login_view.dart';
8 import 'package:http/http.dart' as http;
9
10 class TelaNovaSenha extends StatefulWidget {
11   const TelaNovaSenha({super.key});
12
13   @override
14   State<TelaNovaSenha> createState() => _TelaNovaSenhaState();
15 }
16
17 class _TelaNovaSenhaState extends State<TelaNovaSenha> {
18   TextEditingController codigoController = TextEditingController();
19   TextEditingController emailController = TextEditingController();
20   TextEditingController novaSenhaController = TextEditingController();
21   TextEditingController confirmNovaSenhaController = TextEditingController();
22   String ip_servidor = Utils.ip_servidor;
23
24   Future<void> atualizaSenha(Map<String, dynamic> mapUsuario) async {
25     print("MAPA USUARIO" + mapUsuario.toString());
26     var response = await http.put(
27       Uri.parse('http://$ip_servidor:5000/atualizaSenha'),
28       headers: {
29         'Accept': 'application/json',
30         'content-type': 'application/json',
31       },
32       body: jsonEncode(mapUsuario),
33       encoding: Encoding.getByName('utf-8'),
34     );
35
36     if (response.statusCode == 200) {
37       // Cadastro bem-sucedido
38       ScaffoldMessenger.of(context).showSnackBar(
39         SnackBar(content: Text("Dados atualizados com sucesso!!")),
40       );
41       Navigator.pushReplacement(
42         context,
43         MaterialPageRoute(builder: (context) => LoginView()),
44       );
45     } else {
46       // Erro no cadastro
47       print('Erro ao cadastrar usuário: ${response.body}');
48     }
49   }
50 }
```



```

50
51 @override
52 Widget build(BuildContext context) {
53   String email = ModalRoute.of(context)?.settings.arguments as String;
54
55   return Scaffold(
56     appBar: AppBar(
57       backgroundColor: Colors.blueGrey[800],
58       foregroundColor: Colors.amber,
59       title: const Text('Criar Nova Senha'),
60     ), // AppBar
61     body: Container(
62       padding: const EdgeInsets.all(16),
63       color: Colors.blue[100],
64       child: Column(
65         mainAxisAlignment: MainAxisAlignment.center,
66         children: [
67           Text(
68             'Defina sua nova senha para $email',
69             style: TextStyle(
70               fontSize: 32,
71               fontWeight: FontWeight.bold,
72               color: Colors.blueGrey[800],
73             ), // TextStyle
74           ), // Text
75           const SizedBox(height: 32),
76           TextField(
77             controller: codigoController,
78             keyboardType: TextInputType.number,
79             maxLength: 6,
80             decoration: const InputDecoration(
81               fillColor: Colors.white,
82               filled: true,
83               prefixIcon: Icon(Icons.lock, color: Colors.amber),
84               labelText: 'código de recuperação enviado por e-mail',
85               border: OutlineInputBorder(),
86             ), // InputDecoration
87           ), // TextField
88           const SizedBox(height: 32),
89           TextField(
90             controller: novaSenhaController,
91             obscureText: true,
92             decoration: const InputDecoration(
93               fillColor: Colors.white,
94               filled: true,
95               prefixIcon: Icon(Icons.lock, color: Colors.amber),
96               labelText: 'Nova senha',
97               border: OutlineInputBorder(),
98             ), // InputDecoration
99           ), // TextField
100           const SizedBox(height: 16),
101           TextField(
102             controller: confirmNovaSenhaController,
103             obscureText: true,
104             decoration: const InputDecoration(
105               fillColor: Colors.white,
106               filled: true,
107               prefixIcon: Icon(Icons.lock, color: Colors.amber),
108               labelText: 'Confirme a nova senha',
109               border: OutlineInputBorder(),
110             ), // InputDecoration
111           ), // TextField
112           const SizedBox(height: 16),

```

```

112     const SizedBox(height: 16),
113     ElevatedButton(
114       onPressed: () {
115         if (novaSenhaController.text ==
116             confirmNovaSenhaController.text) {
117           Map<String, dynamic> mapUsuario = {
118             "emailUsuario": email,
119             "novaSenha": novaSenhaController.text,
120             "codigoRecuperacao": int.tryParse(codigoController.text),
121           };
122           atualizaSenha(mapUsuario);
123         } else {
124           ScaffoldMessenger.of(context).showSnackBar(
125             SnackBar(content: Text("As senhas não conferem")),
126           );
127         }
128
129         // Lógica para salvar a nova senha aqui
130       },
131       child: const Text('Salvar nova senha'),
132     ), // ElevatedButton
133   ],
134 ), // Column
135 ), // Container
136 ); // Scaffold
137 }
138 }
139

```

home_view.dart

```
3 import 'dart:convert';
4
5 import 'package:flutter/material.dart';
6 import 'package:funcionarios_autentic_json/services/utils.dart';
7 import 'package:funcionarios_autentic_json/views/login/login_view.dart';
8 import 'package:http/http.dart' as http;
9
10 class HomeView extends StatefulWidget {
11   const HomeView({super.key});
12
13   @override
14   State<HomeView> createState() => _HomeViewState();
15 }
16
17 class _HomeViewState extends State<HomeView> {
18   String ip_servidor = Utils.ip_servidor;
19   List listaFuncionarios = [];
20   dynamic dadosUsuario;
21
22   //FUNÇÃO CADASTRA FUNCIONÁRIO
23   Future<void> cadastraFuncionario(Map<String, dynamic> mapFuncionario) async {
24     var response = await http.post(
25       Uri.parse('http://$ip_servidor:5000/cadastraFuncionario'),
26       headers: {
27         'Accept': 'application/json',
28         'content-type': 'application/json',
29       },
30       body: jsonEncode(mapFuncionario),
31       encoding: Encoding.getByName('utf-8'),
32     );
33
34     if (response.statusCode == 200) {
35       ScaffoldMessenger.of(
36         context,
37       ).showSnackBar(SnackBar(content: Text("Cadastrado com sucesso!!")));
38       setState(() {
39         carregaDados();
40       });
41       Navigator.pop(context);
42     } else {
43       // Erro no cadastro
44       ScaffoldMessenger.of(
45         context,
46       ).showSnackBar(SnackBar(content: Text("Erro ao cadastrar usuário!!")));
47     }
48   }
49
50   //FUNÇÃO CARREGA FUNCIONARIOS NA LISTA
51   Future<void> carregaDados() async {
52     var dadosRecebidos = await http.get(
53       Uri.parse(
54         'http://$ip_servidor:5000/lerFuncionario/${dadosUsuario["usuario"]["id"]}',
55       ),
56     );
57
58     if (dadosRecebidos.statusCode == 200) {
59       print("DADOS RECEBIDOS: " + dadosRecebidos.body);
60
61       setState(() {
62         listaFuncionarios = jsonDecode(dadosRecebidos.body)[0];
63       });
64     }
65   }
66 }
```

```

64     } else {
65         ScaffoldMessenger.of(
66             context,
67         ).showSnackBar(SnackBar(content: Text("Erro ao carregar dados!!")));
68         print("ERRO AO CARREGAR DADOS: " + dadosRecebidos.body);
69     }
70 }
71
72 //FUNÇÃO ATUALIZA FUNCIONÁRIO
73 Future<void> atualizaFuncionario(Map<String, dynamic> mapFuncionario) async {
74     var response = await http.put(
75         Uri.parse('http://$ip_servidor:5000/atualizaFuncionario'),
76         headers: {
77             'Accept': 'application/json',
78             'content-type': 'application/json',
79         },
80         body: jsonEncode(mapFuncionario),
81         encoding: Encoding.getByName('utf-8'),
82     );
83
84     if (response.statusCode == 200) {
85         ScaffoldMessenger.of(
86             context,
87         ).showSnackBar(SnackBar(content: Text("Atualizado com sucesso!!")));
88         setState(() {
89             carregaDados();
90         });
91     } else {
92         // Erro no cadastro
93         ScaffoldMessenger.of(
94             context,
95         ).showSnackBar(SnackBar(content: Text("Erro ao atualizar usuário!!")));
96     }
97 }
98
99 //FUNÇÃO EXCLUIR FUNCIONÁRIO
100 Future<void> excluirFuncionario(Map<String, dynamic> mapFuncionario) async {
101     var response = await http.delete(
102         Uri.parse('http://$ip_servidor:5000/deletaFuncionario'),
103         headers: {
104             'Accept': 'application/json',
105             'content-type': 'application/json',
106         },
107         body: jsonEncode(mapFuncionario),
108         encoding: Encoding.getByName('utf-8'),
109     );
110
111     if (response.statusCode == 200) {
112         ScaffoldMessenger.of(
113             context,
114         ).showSnackBar(SnackBar(content: Text("Excluído com sucesso!!")));
115         setState(() {
116             carregaDados();
117         });
118     } else {
119         ScaffoldMessenger.of(
120             context,
121         ).showSnackBar(SnackBar(content: Text("Erro ao excluir usuário!!")));
122     }
123     Navigator.pop(context);
124 }
125

```

```

126 @override
127 didChangeDependencies() {
128   super.didChangeDependencies();
129   dadosUsuario = ModalRoute.of(context)!.settings.arguments;
130   carregaDados();
131 }
132
133 @override
134 widget build(BuildContext context) {
135   dadosUsuario = ModalRoute.of(context)!.settings.arguments;
136   return Scaffold(
137     appBar: AppBar(
138       title: const Text('Home'),
139       backgroundColor: Colors.blueGrey[800],
140       foregroundColor: Colors.amber,
141     ), // AppBar
142
143     body: Container(
144       padding: const EdgeInsets.all(16),
145       width: double.infinity,
146       height: double.infinity,
147       color: Colors.blue[100],
148       child: Column(
149         mainAxisAlignment: MainAxisAlignment.center,
150         children: [
151           Row(
152             children: [
153               Text(
154                 'Olá ${dadosUsuario["usuario"]["nome"]}',
155                 style: TextStyle(
156                   fontSize: 16,
157                   fontWeight: FontWeight.bold,
158                   color: Colors.blueGrey[800],
159                 ), // TextStyle
160               ), // Text
161               //botao logout
162               IconButton(
163                 icon: const Icon(Icons.logout, color: Colors.redAccent),
164                 onPressed: () {
165                   super.dispose();
166
167                   Navigator.pushReplacement(
168                     context,
169                     MaterialPageRoute(
170                       builder: (context) => const LoginView(),
171                     ), // MaterialPageRoute
172                   );
173                 },
174               ), // IconButton
175             ],
176           ), // Row
177           const SizedBox(height: 16),
178           Expanded(
179             child: ListView.builder(
180               itemCount: listaFuncionarios.length,
181               itemBuilder: (context, index) {
182                 return ListTile(
183                   title: Text(
184                     '${listaFuncionarios[index]["id"]} - ${listaFuncionarios[index]["nomeFuncionario"]}',
185                   ), // Text
186                   subtitle: Text(
187                     'Telefone: ${listaFuncionarios[index]["telefoneFuncionario"]}',
188                   ), // Text

```



```

250         atualizaFuncionario(mapFuncionario);
251         Navigator.pop(context);
252     },
253     child: const Text('Atualizar'),
254 ), // TextButton
255 ],
256 ), // AlertDialog
257 );
258 },
259 ), // IconButton
260 IconButton(
261   icon: const Icon(
262     Icons.delete,
263     color: Colors.redAccent,
264   ), // Icon
265   onPressed: () {
266     // Lógica para excluir o item
267     showDialog(
268       context: context,
269       builder: (context) => AlertDialog(
270         title: const Text('Excluir Funcionário'),
271         content: Text(
272           'Deseja excluir ${listaFuncionarios[index]["nomeFuncionario"]}',
273         ), // Text
274         actions: [
275           TextButton(
276             onPressed: () {
277               Navigator.pop(context);
278             },
279             child: const Text('Cancelar'),
280           ), // TextButton
281           TextButton(
282             onPressed: () {
283               // Lógica para excluir o item
284               Map<String, dynamic> mapFuncionario = {
285                 "idFuncionario":
286                   listaFuncionarios[index]["id"],
287                 "idUserarioFK":
288                   dadosUsuario["usuario"]["id"],
289               };
290               excluirFuncionario(mapFuncionario);
291             },
292             child: const Text('Excluir'),
293           ), // TextButton
294         ],
295       ), // AlertDialog
296     );
297   },
298 ), // IconButton
299 ],
300 ), // Row
301 ); // ListTile
302 },
303 ), // ListView.builder
304 ), // Expanded
305 ],
306 ), // Column
307 ), // Container
308 floatingActionButton: FloatingActionButton(
309   onPressed: () {
310     TextEditingController nomeController = TextEditingController();
311     TextEditingController telefoneController = TextEditingController();

```

```

311   TextEditingController telefoneController = TextEditingController();
312   showDialog(
313     context: context,
314     builder: (context) => AlertDialog(
315       title: const Text('Cadastrar Funcionário'),
316       content: Column(
317         mainAxisAlignment: MainAxisSize.min,
318         children: [
319           TextField(
320             controller: nomeController,
321             decoration: const InputDecoration(LabelText: 'Nome'),
322           ), // TextField
323           TextField(
324             controller: telefoneController,
325             decoration: const InputDecoration(LabelText: 'Telefone'),
326           ), // TextField
327         ],
328       ), // Column
329       actions: [
330         TextButton(
331           onPressed: () {
332             Navigator.pop(context);
333           },
334           child: const Text('Cancelar'),
335         ), // TextButton
336         TextButton(
337           onPressed: () {
338             Map<String, dynamic> mapFuncionario = {
339               "nomeFuncionario": nomeController.text,
340               "telefoneFuncionario": telefoneController.text,
341               "idUserarioFK": dadosUsuario["usuario"]["id"],
342             };
343             cadastraFuncionario(mapFuncionario);
344           },
345           child: const Text('Cadastrar'),
346         ), // TextButton
347       ],
348     ), // AlertDialog
349   );
350 },
351 backgroundColor: Colors.amber,
352 foregroundColor: Colors.blueGrey[800],
+ 353 child: Icon(Icons.add),
354 ), // FloatingActionButton
355 ); // Scaffold
356 }
357 }
358

```